Git Analyzer Architecture Handbook

Core Backend Design Patterns, Environment Sync, and Data Flow Strategies

Developer: Lokesh S. | Reg No: 23BPS1073 | Technical Reference Manual

1. The 4-Tier Backend Pipeline

In modern enterprise Spring Boot applications, complexity is managed by breaking the codebase down into specialized layers. This strict separation of concerns prevents code degradation, enhances testability, and guarantees clean horizontal scalability.

1. Controller Layer (The API Gateway / Receptionist)
Exposes web REST endpoints (e.g., `@RestController` mapped to `/api/repositories/analyze`). Its primary responsibility is handling incoming HTTP network requests, unwrapping the JSON request bodies into Java Data Transfer Objects (DTOs), executing basic syntactic verification, and returning descriptive HTTP state responses back to the frontend.

2. Service Layer (The System Brains / Business Logic)
Sits securely between the interface layer and data persistence. This is where the core algorithms and operational rules of your application live. It manages authorization validations, checks user request thresholds, coordinates background transaction flows, and executes external network triggers—such as making outbound webhook calls to your n8n workflow engine.

3. Repository Layer (The Data Access Bridge / Driver)
Interfaces directly with the physical storage ecosystem. By extending Spring Data JPA's `JpaRepository`, this interface layer completely abstracts away manual database drivers. It handles all reading, updating, inserting, and deleting routines using elegant object-oriented method calls, entirely eliminating the need to write fragile SQL strings by hand.

4. Entity Layer (The Database Blueprint / Car)
Plain Old Java Objects (POJOs) heavily annotated with persistence annotations like `@Entity` and `@Table`. Each entity class acts as a precise structural twin to an existing table inside your PostgreSQL container, informing the backend exactly how data structures map between memory blocks and physical tables.

2. The Core Paradigm: Entities vs. Repositories

A frequent conceptual hurdle is understanding why a repository requires a separate entity layer if it already executes data operations. They operate as an indivisible structural team: **the entity defines the data map,**

while the repository provides the action framework. Without the entity blueprint, the repository cannot build data blocks because it is completely blind to your schema layouts.

> **📦 The Amazon Shipping Analogy**
>
> Think of your architecture as a logistics pipeline. The **Repository** behaves exactly like a **Delivery Truck**; it contains the operational logic required to drive to a destination, interact with the physical property, and fulfill a delivery. The **Entity** represents the **Cardboard Shipping Box**; it is structurally built with explicit compartments (columns) designed to safely wrap your payloads (ID, URL, timestamps). If you command the Delivery Truck (Repository) to execute a drop-off without giving it an explicitly packed Shipping Box (Entity), the truck has no cargo to carry.

This coupling is explicitly declared when configuring the repository interface layer, defining exactly which entity shape the truck is built to handle:

```
public interface UserRepository extends JpaRepository<User, UUID> {
    // Tells Spring Boot to generate a truck optimized exclusively to process "User" entity
packages
    Optional<User> findByFirebaseUid(String firebaseUid);
}
```

3. Active Environment Infrastructure Sync

The system integrates an atomic application framework communicating dynamically with isolated environments inside a Docker container ecosystem. The following landscape represents your verified runtime configuration:

INFRASTRUCTURE ITEM	PORT MAPPING	CORE DIRECTIVE / RULE	OPERATIONAL ROLE
PostgreSQL Database	`5432`	Docker Isolated Instance	Secures schemas for user metadata, repository tracking, and high-dimensional vector chunks.
Spring Boot Backend	`8080`	`ddl-auto=validate`	Acts as the core processing logic engine. **Validate** enforces verification against your existing tables, completely preventing runtime wipes.
Maven Ecosystem	N/A	`./mvnw` Wrapper Script	Ensures workspace uniformity across compiler runtimes, automatically resolving JPA, Web, and Lombok dependencies.

4. Repository Analysis Data Flow Strategies

When processing GitHub URLs submitted by your interface layer, your backend separates activities into two functional modes depending on the depth of analytical context requested:

Strategy A: The "Quick View" (Direct Pass-Through)

Designed for rapid, zero-overhead user feedback. When a user queries a public codebase, the application executes an external API request straight to `https://api.github.com/repos/{owner}/{repo}/commits`. The incoming JSON array maps directly to the UI view for commit logs or stats without allocating server disk space, creating entries in your database container, or generating token processing costs.

Strategy B: The "Deep AI Analysis" (Persistent Semantic Indexing)

This is the primary feature pipeline of your Git Analyzer platform. When confirmed, a comprehensive architectural pipeline executes across your backend components:

- **Ingestion:** The Controller receives the URL and hands execution over to the Service layer.
- **Automation Trigger:** The Service layer makes an HTTP call to your **n8n engine** via a secure webhook link.
- **Cloning & Extraction:** n8n securely clones the target codebase down to a temporary disk volume and extracts deep commit logs via internal Git hooks.
- **Vectorization:** The files are broken down into logical text blocks and processed through OpenAI's embedding model to generate complex 1536-dimensional vector arrays.

- **Data Storage:** The extracted commit data maps to your `repositories` table, while the vector embeddings are written to the `code_chunks` table using relational foreign key constraints (`user_id`), ready to handle advanced semantic AI searches.